

Aprendizaje Computacional

Titulación: Grado en Ingeniería Informática (4ºA)

Curso: 2025-2026

Trabajo: Digit Recognition

Autores: Ángel Campos Salido, Steven Rodríguez Santos, Roberto Ramos Trillo y Sophia Requião de Almeida

ÍNDICE

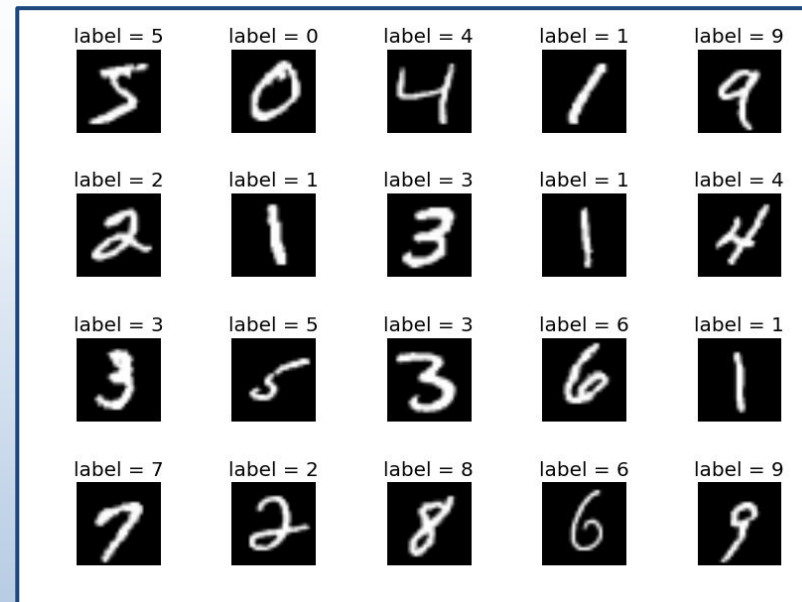
- 1. INTRODUCCIÓN**
- 2. SVM (SUPPORT VECTOR MACHINE)**
- 3. ÁRBOLES DE DECISIÓN**
- 4. RANDOM FOREST**
- 5. PERCEPTRÓN MULTICAPA**
- 6. CONCLUSIÓN**

INTRODUCCIÓN

EXPLICACIÓN DEL EJERCICIO

Nos encontramos con una base de datos de 70000 imágenes de dígitos con dimensiones de 28x28 en escala de grises, cada una etiquetada con el número al que corresponde.

Nuestro objetivo es usar estas imágenes para entrenar un modelo que pueda clasificar imágenes similares de forma precisa.



PREPARACIÓN DE LOS DATOS

Antes de usar los modelos, debemos preprocesar los datos:

1. Normalizamos los valores de cada píxel para que estén entre 0 y 1.
2. Convertimos cada patrón de una matriz de 28x28 a un vector de 784 características.
3. Tomamos 1000 patrones para entrenamiento y 250 para test para hacer más ligera la selección de modelo

INTRODUCCIÓN

MODELOS

Nuestro objetivo es encontrar el modelo que mejor se ajuste a nuestras necesidades, por lo que probaremos distintos modelos para ver cuál es el más acertado

Estos modelos son:

- SVM
- Decision Trees
- Random Forest
- Multilayer Perceptron

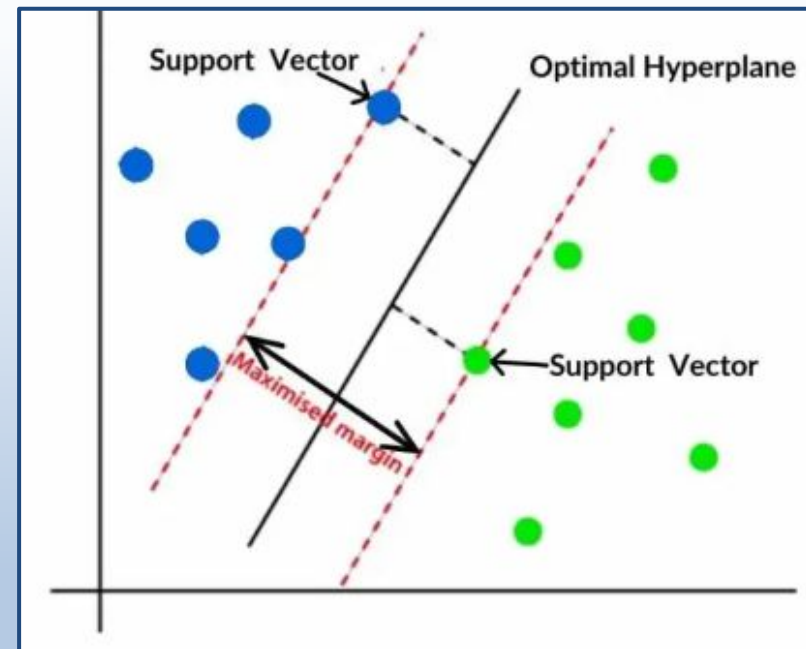


SVM (SUPPORT VECTOR MACHINE)

EXPLICACIÓN DEL MODELO

Es un algoritmo de aprendizaje supervisado para clasificación que encuentra el hiperplano óptimo que separa clases en el espacio de características.

Puede manejar datos no lineales usando un kernel adecuado (en este caso usamos un kernel radial)



SVM (SUPPORT VECTOR MACHINE)

PREPROCESAMIENTO

```
# cargamos la base de datos
mnist <- read_mnist()

X_train <- mnist$train$images
X_test <- mnist$test$images
y_train <- mnist$train$labels
y_test <- mnist$test$labels

# normalizamos los valores para que estén entre 0 y 1
X_train <- X_train / 255
X_test <- X_test / 255

#seleccionamos una muestra de los datos para train y test
#train 1000, test 250 (80/20)
set.seed(123)
train_indexes <- sample(1:nrow(X_train), 1000)
test_indexes <- sample(1:nrow(X_test), 250)

X_train_small <- X_train[train_indexes,]
X_test_small <- X_test[test_indexes,]
y_train_small <- y_train[train_indexes]
y_test_small <- y_test[test_indexes]
```

SVM (SUPPORT VECTOR MACHINE)

BÚSQUEDA DE HIPERPARÁMETROS

```
# 3) Búsqueda de hiperparámetros

#buscamos hiperparámetros
tuned <- e1071::tune(
  METHOD = svm,
  train.x = X_train_small,
  train.y = as.factor(y_train_small),
  ranges = list(
    cost = c(0.1, 1, 10),
    gamma = c(0.001, 0.01, 0.1)
  )
)

#encontramos los mejores hiperparámetros
print(tuned$best.parameters)
print(tuned$best.performance)
```

```
> print(tuned$best.parameters)
  cost gamma
6   10 0.01
> print(tuned$best.performance)
[1] 0.079
```


SVM (SUPPORT VECTOR MACHINE)

ENTRENAMIENTO DEL MODELO FINAL

```
# 4) Entrenamiento de modelo final

svm_model <- svm(
  x = X_train_small,
  y = as.factor(y_train_small),
  kernel = "radial",
  cost = tuned$best.parameters$cost,
  gamma = tuned$best.parameters$gamma
)

summary(svm_model)
```

Parameters:

SVM-Type: C-classification

SVM-Kernel: radial

cost: 10

Number of Support Vectors: 701

(60 92 55 79 83 53 68 70 82 59)

Number of Classes: 10

SVM (SUPPORT VECTOR MACHINE)

EVALUACIÓN DEL MODELO FINAL

```
#predicción del modelo
pred <- predict(svm_model, X_test_small)

#matriz de confusión
matrix <- table(
  Predicted = pred,
  Actual = y_test_small
)
print(matrix)

#calculamos accuracy
accuracy <- sum(diag(matrix)) / sum(matrix)
cat("Precisión del modelo SVM:", accuracy, "\n")
```

```
> print(matrix)
      Actual
Predicted 0 1 2 3 4 5 6 7 8 9
0 28 0 1 0 0 0 1 0 0 0
1 0 26 0 0 0 0 0 0 0 0
2 0 0 25 0 0 0 0 1 1 0
3 0 0 1 19 0 0 0 0 1 1
4 0 0 1 0 29 0 0 0 0 1
5 0 0 0 1 0 25 0 0 1 0
6 0 0 0 0 0 0 22 0 0 0
7 0 0 0 1 0 0 0 25 0 0
8 0 0 0 0 0 0 0 0 19 0
9 0 0 0 0 0 0 0 0 0 20

>
> #calculamos accuracy
> accuracy <- sum(diag(matrix)) / sum(matrix)
> cat("Precisión del modelo SVM:", accuracy, "\n")
Precisión del modelo SVM: 0.952
```

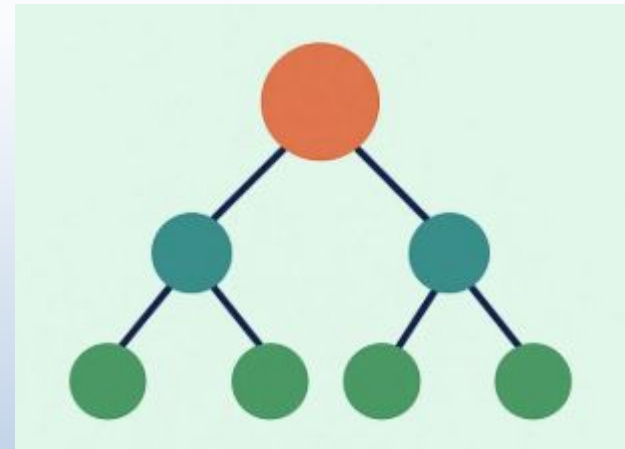
Precisión del 95,2%

ÁRBOLES DE DECISIÓN

EXPLICACIÓN DEL MODELO

Es un algoritmo de aprendizaje supervisado que crea un modelo en forma de estructura de árbol con decisiones basadas en atributos.

Aquí usamos boosting para combinar varios árboles débiles para crear un modelo más fuerte



Cargar datos de mnist

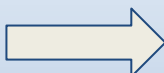
```
mnist <- read_mnist()
set.seed(123)
idx <- sample(nrow(mnist$train$images), 1250)
mnist$train$images <- mnist$train$images[idx, ]
mnist$train$labels <- mnist$train$labels[idx]

mnist_dataset <- as.data.frame(mnist$train$images)
mnist_dataset$label <- mnist$train$label
mnist_dataset$label <- as.factor(mnist_dataset$label)
mnist_dataset # dataset final

idx_train <- sample(1250, 1000)
dtrain <- mnist_dataset[idx_train,]
dtest <- mnist_dataset[-idx_train,]
```

Entrenamiento del modelo básico

```
##### MODELO BASICO #####  
modelo <- rpart(label ~ ., data=dtrain, method="class")  
ac1 <- calcular_accuracy_rpart(modelo, dtest)  
pred <- predict(modelo, newdata=dtest, type="class")  
ac1
```

Modelo básico sin modificar
hiper parámetros  Accuracy de 0.672

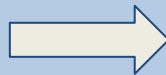
Entrenamiento del modelo con un grid

```
##### MODELO CON GRID SEARCH #####

# Para validacion cruzada
control <- trainControl(method="cv", number=8)
# Grid para parametros a probar
grid <- expand.grid(cp = seq(0.01, 0.1, by = 0.01))

mejor_modelo <- NULL
mejor_acc <- 0
iter <- 0
for (ms in c(1,3,5,8)) {
  for (md in c(15, 20, 30)) {
    control_rpart <- rpart.control(minsplit = ms, maxdepth = md)
    modelo <- train(label ~., data=dtrain, method="rpart", trControl=control, tuneGrid=grid, control=control_rpart)
    acc <- calcular_accuracy_caret(modelo, dtest)
    print(paste("Iter:", iter))
    iter <- iter + 1
    if (acc > mejor_acc) {
      mejor_acc <- acc
      mejor_modelo <- modelo
    }
  }
}
mejor_acc
```

Búsqueda con
un grid de hiper
parámetros



Accuracy de 0.68

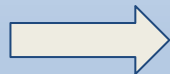
Entrenamiento del modelo con Adaboost

```
##### MODELO BOOSTING #####

modelo_boosting <- boosting(label ~., data=dtrain, boos=TRUE, mfinal=50,
                             control = rpart.control(
                               cp = 0.000,
                               maxdepth = 30,
                               minbucket = 1
                             ))

pred_boost <- predict(modelo_boosting, newdata=dtest)
ac_boost <- mean(pred_boost$class == dtest$label)
ac_boost
```

Algoritmo
adaboost



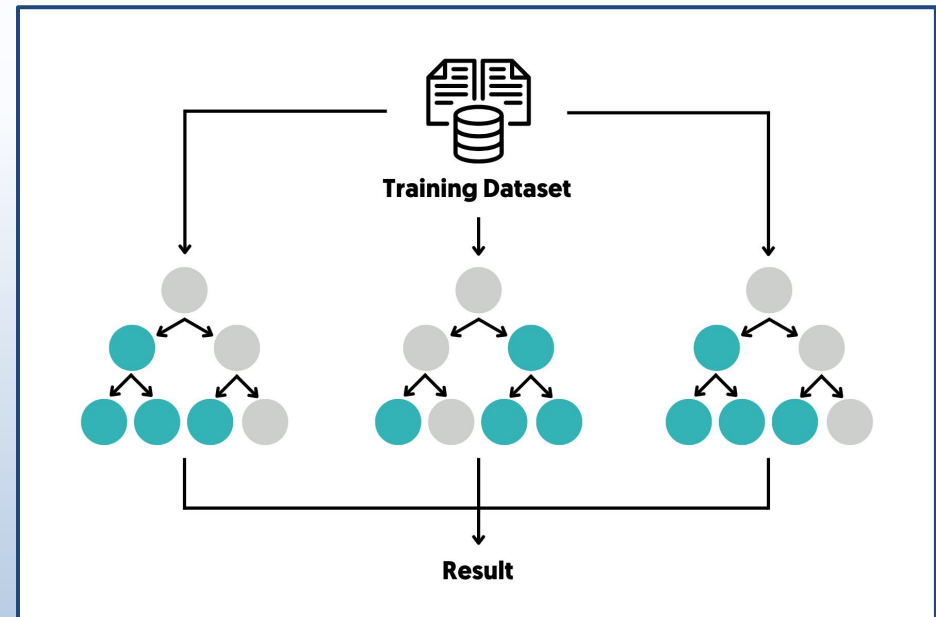
Accuracy de 0.904 ✨

RANDOM FOREST

EXPLICACIÓN DEL MODELO

Es un algoritmo de ensamblado que combina múltiples árboles de decisión entrenados en paralelo.

Está compuesto por un bosque de árboles que votan democráticamente.



RANDOM FOREST

Cargando datos en memoria

```
library(randomForest)
library(caret)

# 1. Cargar datos -----

train <- read.csv("C:/Users/rober/Downloads/train.csv")
test  <- read.csv("C:/Users/rober/Downloads/test.csv")

# Aseguramos que la etiqueta es factor
train$label <- factor(train$label)

# 2. Submuestreo: 1000 train, 250 test interno -----

set.seed(123)

# Elegimos 1250 ejemplos aleatorios del train original
idx_total <- sample(seq_len(nrow(train)), 1250)
data_small <- train[idx_total, ]

# De esos 1250, cogemos 1000 para entrenar y 250 para test interno
idx_train <- sample(seq_len(nrow(data_small)), 1000)
train_data <- data_small[idx_train, ]
test_data  <- data_small[-idx_train, ] # el resto (250 filas)

# Aseguramos factor en las etiquetas
train_data$label <- factor(train_data$label)
test_data$label  <- factor(test_data$label)
```

RANDOM FOREST

Preprocesamiento de los datos

```
# 3. Normalización de píxeles [0,1] -----  
  
# Entrenamos el preprocesado SOLO sobre los datos de entrenamiento  
preproc <- preProcess(train_data[ , -1], method = "range")  
  
# Aplicamos la transformación  
x_train <- predict(preproc, train_data[ , -1])  
x_test  <- predict(preproc, test_data[ , -1])  
  
# Nos aseguramos de que x_test tiene las mismas columnas que x_train  
x_test <- x_test[ , colnames(x_train)]  
  
y_train <- train_data$label  
y_test  <- test_data$label
```

RANDOM FOREST

Entrenamiento del modelo

```
# 4. Entrenamiento del Random Forest -----  
  
set.seed(123)  
rf_model <- randomForest(  
  x      = x_train,  
  y      = y_train,  
  ntree  = 300,  
  mtry   = 30,  
  importance = TRUE  
)  
  
print(rf_model)
```

RANDOM FOREST

Validación del modelo

```
# 5. Evaluación en el test interno (250 ejemplos) -----
pred_test <- predict(rf_model, x_test)

conf_mat <- confusionMatrix(pred_test, y_test)
print(conf_mat)

accuracy <- conf_mat$overall["Accuracy"]
cat("Accuracy en el test interno (250 ejemplos):", accuracy, "\n")
```

```
> print(conf_mat)
Confusion Matrix and Statistics
```

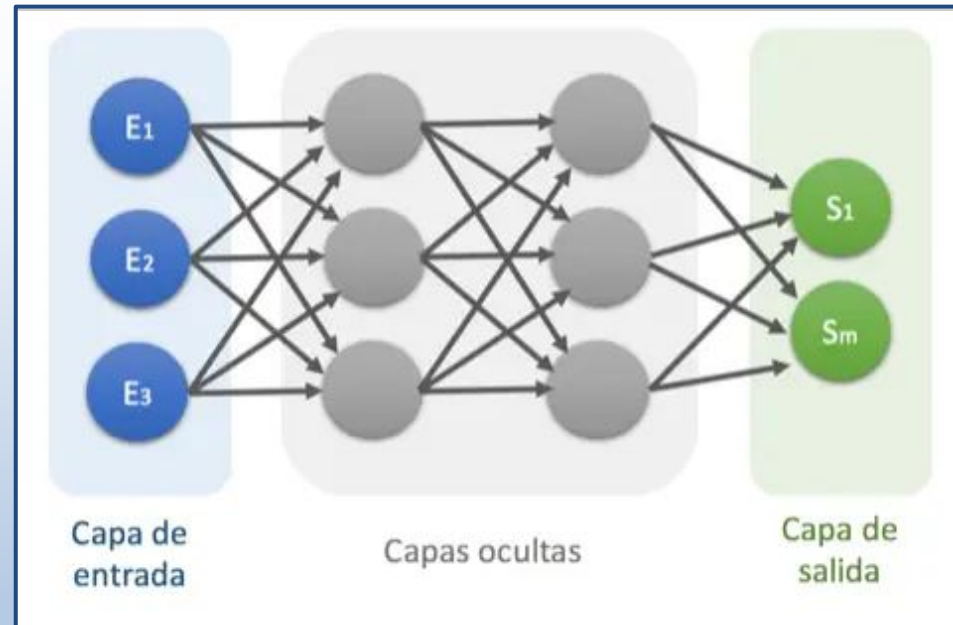
	Reference									
Prediction	0	1	2	3	4	5	6	7	8	9
0	25	0	0	1	0	0	0	0	0	0
1	0	18	1	0	0	0	0	1	0	0
2	0	1	26	1	0	1	0	1	0	1
3	0	0	0	16	0	0	0	0	0	0
4	0	0	0	0	26	1	0	1	0	2
5	0	0	0	3	0	18	0	0	1	0
6	0	0	0	0	1	1	19	0	0	0
7	0	0	1	0	0	0	0	32	0	0
8	2	0	0	2	0	0	1	0	19	0
9	0	0	0	0	1	0	0	2	0	24

Accuracy : 0.892

PERCEPTRÓN MULTICAPA

EXPLICACIÓN DEL MODELO

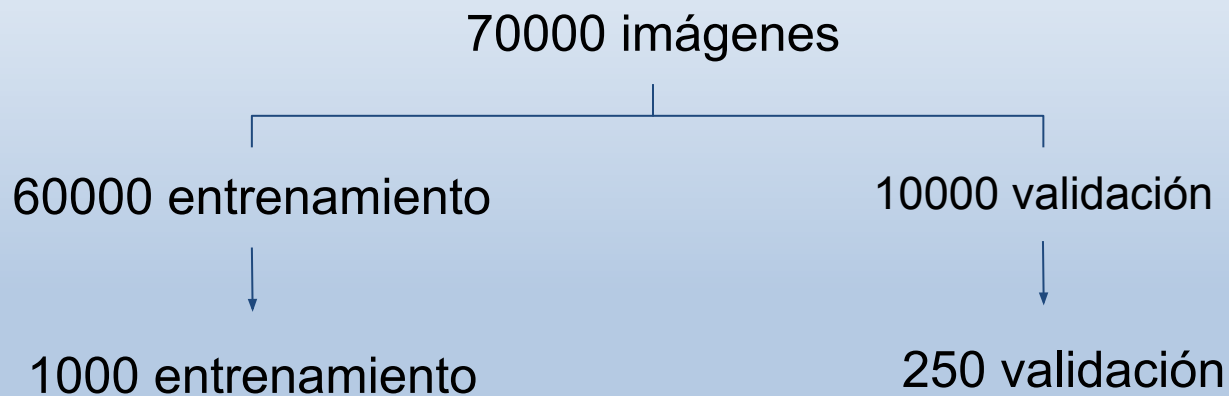
Es una red neuronal artificial con múltiples capas, lo cual le hace capaz de aprender relaciones no lineales complejas mediante la propagación hacia atrás



PERCEPTRÓN MULTICAPA

Cargando datos en memoria:

```
# =====  
# 2. CARGA Y PREPROCESAMIENTO DE LOS DATOS  
# =====  
mnist <- dataset_mnist()  
  
# Selección de las primeras 1000 imágenes de entrenamiento y 250 de prueba  
x_train <- mnist$train$x[1:1000, , ]  
y_train <- mnist$train$y[1:1000]  
x_test <- mnist$test$x[1:250, , ]  
y_test <- mnist$test$y[1:250]
```



Preprocesamiento de datos:

```
# Redimensionamiento de las imágenes: 28x28 se convierte en un vector de 784 características y normalización
x_train <- array_reshape(x_train, c(nrow(x_train), 784)) / 255
x_test  <- array_reshape(x_test,  c(nrow(x_test), 784)) / 255

# Codificación de las etiquetas en formato one-hot (vectores binarios)
y_train <- to_categorical(y_train, num_classes = 10)
y_test  <- to_categorical(y_test,  num_classes = 10)
```

- Redimensionamiento y aplanamiento (*Reshape*)
 - Normalización (Escala 0 a 1)
 - Codificación *One-Hot* de las Etiquetas

Reducción de dimensionalidad:

```
# =====  
# 3. ANÁLISIS DE COMPONENTES PRINCIPALES (PCA)  
# =====  
# Aplica el PCA a los datos de entrenamiento  
pca <- prcomp(x_train, center = TRUE, scale. = FALSE)  
  
# Calcula la varianza explicada por cada componente principal  
var_explicada <- pca$sdev^2 / sum(pca$sdev^2)  
var_acumulada <- cumsum(var_explicada)  
  
# Determina el número mínimo de componentes necesarios para explicar el 95% de la varianza  
n_comp_95 <- which(var_acumulada >= 0.95)[1]  
  
# Reducción de dimensionalidad: selecciona solo los primeros 'n_comp_95' componentes principales  
x_train <- as.matrix(pca$x[, 1:n_comp_95])  
x_test  <- predict(pca, x_test)[, 1:n_comp_95]
```

784 características -> computacionalmente costoso y propenso a *overfitting*

Solucion: Análisis de Componentes Principales (PCA)

Grid search:

```
# Hiperparámetros a probar
layers_to_test <- c(1, 2, 3) . . . . . # Número de Capas Ocultas
initial_units_to_test <- c(256, 128) . # Unidades para la primera capa
dropout_rates_to_test <- c(0.2, 0.4) . # Tasas de Dropout

results_full_grid <- list()
counter <- 1

for (L in layers_to_test) {
  for (units in initial_units_to_test) {
    for (rate in dropout_rates_to_test) {
```

Capas Ocultas (1, 2, 3)
Unidades Iniciales (256, 128)
Tasa de *Dropout* (0.2 (20%), 0.4 (40%))

Compilación del modelo

```
# 2. Compile el Modelo
model_grid_test %>% compile(
  optimizer = 'adam',
  loss = 'categorical_crossentropy',
  metrics = c('accuracy')
)
```

Función de pérdida: entropía cruzada

Métrica de evaluación: precisión (accuracy)

Entrenamiento y evaluación del modelo

```
# 3. Entrenar el Modelo
model_grid_test %>% fit(
  x_train, y_train,
  epochs = 25,
  batch_size = 128,
  validation_split = 0.2,
  verbose = 0
)

# 4. Evaluar el Modelo
score <- model_grid_test %>% evaluate(x_test, y_test, verbose = 0)
```

Se almacenan de cada intento: loss y accuracy

Validación del mejor modelo:

Accuracy en el test interno: 0.916
Loss en el test interno: 0.3175

CONCLUSIÓN

VALORACIÓN DE LOS MODELOS

El accuracy obtenido para cada modelo es el siguiente:

- SVM: 95.4%
- Árbol de decisión simple: 68%
- Árbol de decisión con Adaboost: 90.4%
- Random Forest: 89.2%
- Perceptrón Multicapa: 91,6%

Árbol de Decisión <<<< Random Forest < Árbol de Decisión Adaboost < Perceptrón Multicapa < SVM

CONCLUSIÓN

CONCLUSIÓN FINAL

A partir de los resultados anteriores, podemos concluir que SVM es el mejor modelo por tener mayor precisión... lo cual es sospechoso porque el Perceptrón Multicapa debería ser mejor para clasificación de imágenes.

Lo más probable es que SVM haya sacado un accuracy muy alto por overfitting, por lo que hay que tomar nuestros resultados con escepticismo.